Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Методы защиты информации»

ОТЧЕТ

к лабораторной работе №5

на тему:

«ХЕШ-ФУНКЦИЯ»

БГУИР 6-05-0612-02 74

Выполнил студент группы 353504
ЛЕВШУКОВ Дмитрий Александрович

Проверил ассистент каф. Информатики
ПОЛЯКОВ Кирилл Борисович

Минск 2026

СОДЕРЖАНИЕ

Введение 3
1 Выполнение работы 5
Вывод 8
Приложение А (обязательное) Исходный код программы 9

ВВЕДЕНИЕ

Хеш-функции получили широкое распространение в разнообразных алгоритмах быстрого поиска информации. Однако с появлением криптографии у них появилась вторая, ничуть не меньшая, область применения.

Хеш-функцией называется необратимое преобразование данных, обладающее следующими свойствами:

– на вход алгоритма преобразования может поступать двоичный блок данных произвольной длины;

– на выходе алгоритма получается двоичный блок данных фиксированной длины;

– значения на выходе алгоритма распределяются по равномерному закону по всему диапазону возможных результатов;

– при изменении хотя бы одного бита на входе алгоритма его выход значительно меняется: в идеальном случае инвертируется произвольная половина бит.

Основное, но не единственное, предназначение хеш-функций в криптографии – вычисление «неподделываемых» контрольных сумм документов.

Действительно, если для алгоритма хеширования выполняются требования невозможности подобрать иной документ с той же хеш-суммой и невозможности подобрать два документа с произвольной одинаковой хеш-суммой, то хеш-сумма становится уникальной характеристикой документа.

В алгоритме ГОСТ 34.11 используются следующие преобразования:

X-преобразование. На вход функции X подаются две последовательности длиной 512 бит каждая, выходом функции является XOR этих последовательностей.

S-преобразование. Функция S является обычной функцией подстановки. Каждый байт из 512-битной входной последовательности заменяется соответствующим байтом из таблицы подстановок π.

P-преобразование. Функция перестановки. Для каждой пары байт из входной последовательности происходит замена одного байта другим.

L-преобразование. Представляет собой умножение 64-битного входного вектора на бинарную матрицу A размерами 64x64.

Основным элементом любой хеш-функции является функция сжатия. Пусть h, N и m – 512-битные последовательности. Для вычисления функции g(N, m, h) необходимо проделать следующие шаги: вычислить значение K = h $\oplus$ N, присвоить значение K = S(K), присвоить значение K = P(K), присвоить значение K = L(K), вычислить t = E(K, m), присвоить значение t = h $\oplus$ t, вычислить значение G = t $\oplus$ m, вернуть G в качестве результата вычисления функции g(N, m, h).

Функция E(K, m) выполняет следующие действия: вычисление значения state = K $\oplus$ m, для i=0 по 11 выполнить: присвоение значения state = S(state); присвоение значения state = P(state); присвоение значения state = L(state);

вычисление K=KeySchedule(K, i); присвоение значения state = state $\oplus$ K. Вернуть state в качестве результата.

Функция KeySchedule(K, i) отвечает за формирование временного ключа K на каждом раунде функции E(K, m) и производит следующие вычисления: присвоение значений K = K $\oplus$ C[i], K = S(K), K = P(K), K = L(K), возврат K в качестве результата функции.

Вычисление хеш-функции включает следующие шаги:

Для любого входного сообщения M:

1 Присвоить начальные значения внутренних переменных: для хеш-функции с длиной выхода 512 бит h=iv=0x00^{64}, для хеш-функции с длиной выхода 256 бит h=iv=0x01^{64}, N = 0^{512}, $\Sigma = 0^{512}$.

2 Проверить следующее условие:

Длина сообщения M<512. Если условие выполняется перейти к пункту 3). В противном случае выполнить последовательность вычислений: m – последние 512 бит сообщения M, h = g(N, m, h), N = (N + 512) mod 2^{512}, Σ = (Σ + m) mod 2^{512}.

Обрезать M, убрав последние 512 бит и перейти к шагу 2 .

3 Произвести дополнение сообщения M до длины в 512 бит по следующему правилу: m = $0^{511-|M|}||1||$M, где |M| – длина сообщения M в битах.

Вычислить h = g(N, m, h).

Вычислить N = (N + |M|) mod 2^{512}.

Вычислить Σ = (Σ + m) mod 2^{512}.

Вычислить h = g(0, h, N).

Вычислить h = g(0, h, Σ).

Для хеш-функции с длиной выхода в 512 бит возвращаем h в качестве результата. Для функции с длиной выхода 256 бит возвращаем MSB 256 (h).

Алгоритм SHA-1 предложен Институтом Стандартизации США NIST как стандарт хеширования в гражданской криптографии. Этот алгоритм был призван дать еще больший запас прочности к криптоатакам.

SHA-1 реализует хеш-функцию, построенную на идее функции сжатия. Входами функции сжатия являются блок сообщения длиной 512 бит и выход предыдущего блока сообщения.

Выход представляет собой значение всех хеш-блоков до этого момента. Иными словами, хеш блока Mi равен hi = f (Mi , hi−1). Хеш-значением всего сообщения является выход последнего блока.

Алгоритм получает на входе сообщение максимальной длины 2^{64} бит и создает в качестве выхода дайджест сообщения длиной 160 бит.

Задание:

1 Изучить теоретические сведения.

2 Реализовать самостоятельно (без использования готовых библиотек и функций) программное средство контроля целостности сообщений с помощью вычисления хеш-функции и алгоритма ГОСТ 34.11 и SHA-1.

1 ВЫПОЛНЕНИЕ РАБОТЫ

Общий алгоритм SHA-1 представлен в виде блок-схемы на рисунке 1.1.

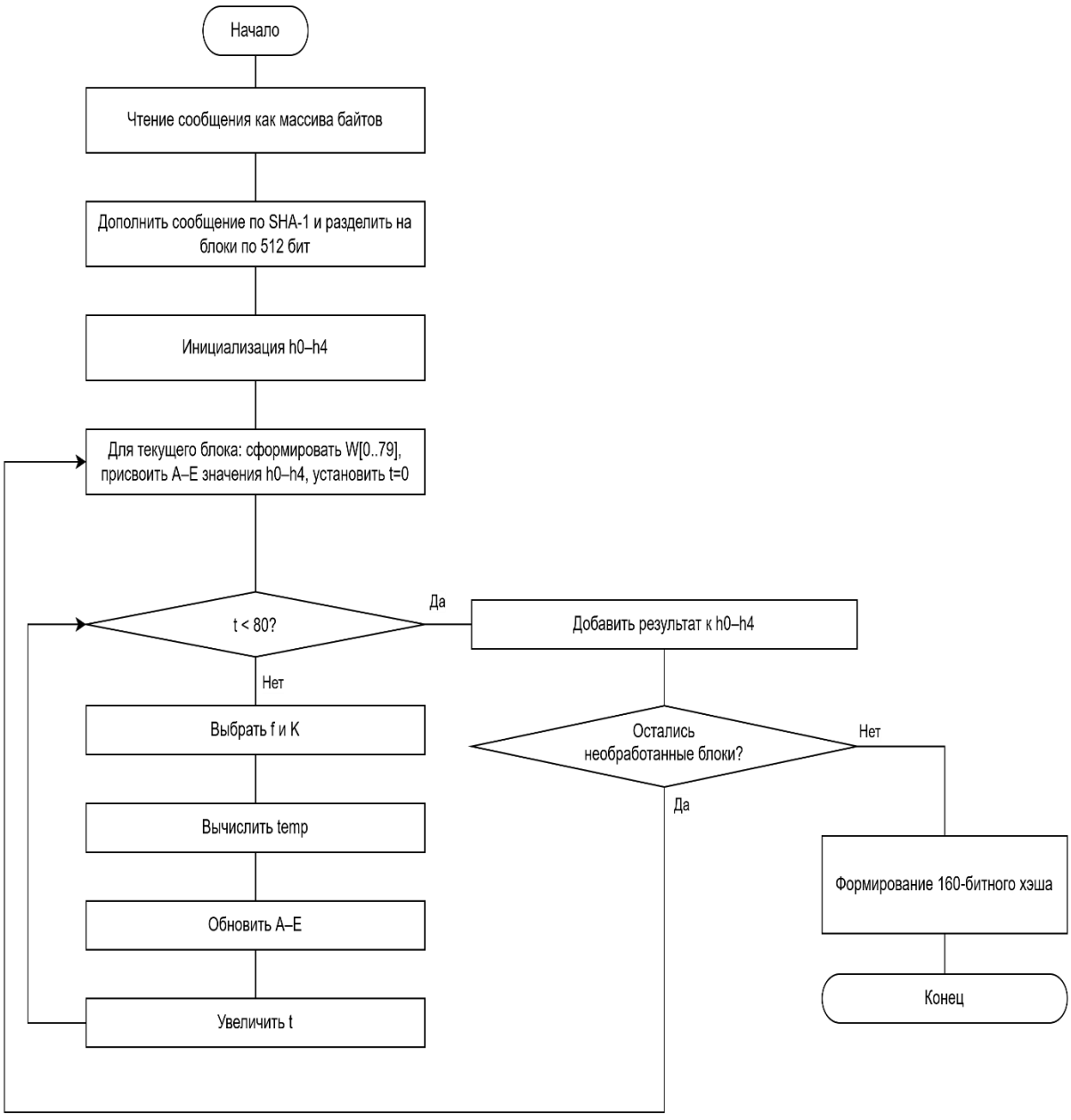


Рисунок 1.1 – Блок-схема алгоритма SHA-1

Общий алгоритм ГОСТ 34.11-2012 представлен в виде блок-схемы на рисунке 1.2.

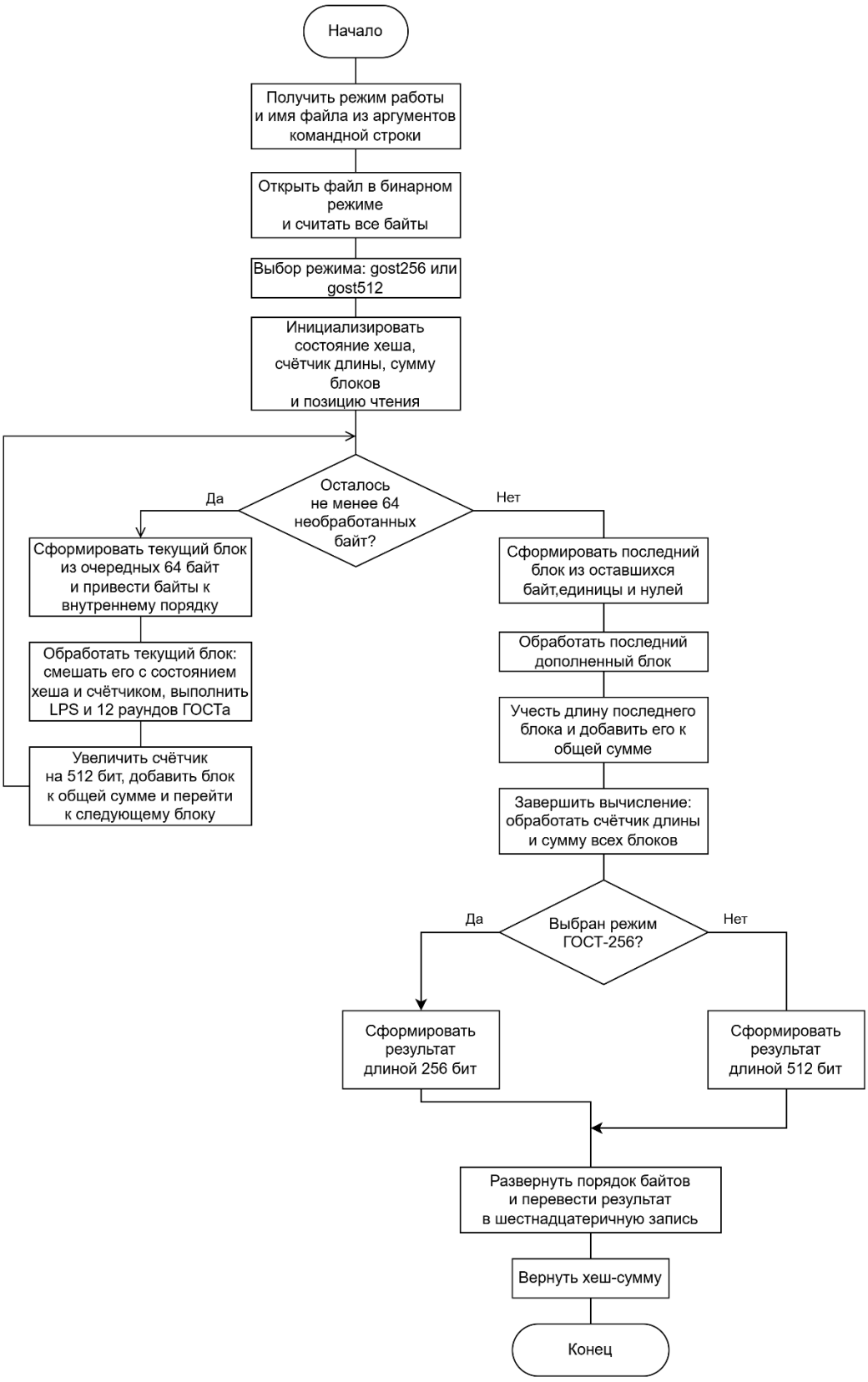


Рисунок 1.2 – Блок-схема алгоритма ГОСТ 34.11-2012

Для проверки работы разработанного алгоритма подается файл test.txt, содержимое которого представлено на рисунке 1.3.

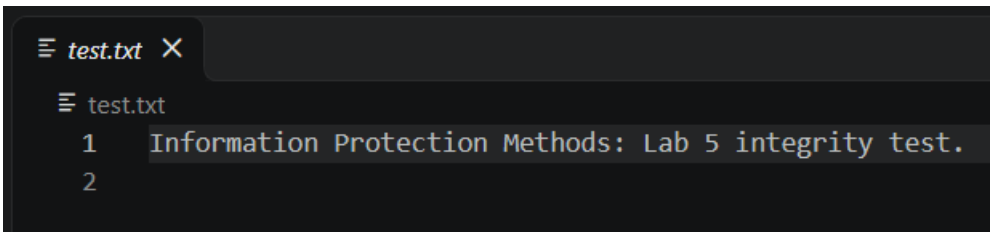


Рисунок 1.3 – Содержимое файла test.txt

Результат выполнения работы программы в режиме SHA-1 представлен на рисунке 1.4.

```
PS C:\Prog\BSUIR-Labs\Information Protection Methods\Lab 5> .\build\release\Lab5.exe sha1 .\test.txt
SHA-1: 76fffbc57664616629e4e64f676df49010ad02d4
```

Рисунок 1.4 – Результат выполнения работы программы в режиме SHA-1

Результат выполнения работы программы в режиме GOST256 представлен на рисунке 1.5.

```
PS C:\Prog\BSUIR-Labs\Information Protection Methods\Lab 5> .\build\release\Lab5.exe gost256 .\test.txt
GOST 34.11-2012 (256-bit): 149ea4b36d337c3685f7d8f3e533e94679ea7e004bf926ce1ca864b79a58071e
```

Рисунок 1.5 – Результат выполнения работы программы в режиме GOST256

Результат выполнения работы программы в режиме GOST512 представлен на рисунке 1.6.

```
PS C:\Prog\BSUIR-Labs\Information Protection Methods\Lab 5> .\build\release\Lab5.exe gost512 .\test.txt
GOST 34.11-2012 (512-bit): a3d2eb0a3fc983030324f944ca74e12083f6f68f487415449428f71b659dbcd0f2dd596f6cb0ddc4c07e1b9f15482c54f6db7c768b5270310dff4e6a5efaa559
```

Рисунок 1.6 – Результат выполнения работы программы в режиме GOST512

ВЫВОД

В ходе лабораторной работы были изучены принципы работы криптографических хеш-функций SHA-1 и ГОСТ Р 34.11-2012. Были рассмотрены подготовка входного сообщения, его обработка блоками, формирование промежуточного состояния и получение хеш-суммы фиксированной длины.

В результате была разработана программа на C++, самостоятельно реализующая алгоритмы SHA-1 и ГОСТ Р 34.11-2012 без использования готовых функций хеширования и криптографических библиотек. Программа считывает файл в бинарном режиме и поддерживает вычисление SHA-1 длиной 160 бит, ГОСТ-256 длиной 256 бит и ГОСТ-512 длиной 512 бит.

При реализации были использованы операции XOR, циклические сдвиги, сложение с переносом, таблицы подстановок и перестановок, а также функции сжатия и раундовое перемешивание данных. Полученная хеш-сумма может применяться для контроля целостности файла: если содержимое изменилось, повторно вычисленный хеш будет отличаться от исходного. При этом хеш-функция обнаруживает факт изменения данных, но не восстанавливает и не исправляет повреждённый файл.

ПРИЛОЖЕНИЕ А
(обязательное)
Исходный код программы

```
#include <algorithm>
#include <array>
#include <cstdint>
#include <fstream>
#include <iostream>
#include <limits>
#include <stdexcept>
#include <string>
#include <string_view>
#include <vector>

using Block = std::array<std::uint8_t, 64>;

std::uint32_t rotateLeft(std::uint32_t value, unsigned int shift)
{
    return (value << shift) | (value >> (32 - shift));
}

constexpr std::array<std::uint8_t, 256> PI = {
    252, 238, 221, 17, 207, 110, 49, 22, 251, 196, 250, 218, 35, 197, 4,
77,
    233, 119, 240, 219, 147, 46, 153, 186, 23, 54, 241, 187, 20, 205, 95,
193,
    249, 24, 101, 90, 226, 92, 239, 33, 129, 28, 60, 66, 139, 1, 142, 79,
    5, 132, 2, 174, 227, 106, 143, 160, 6, 11, 237, 152, 127, 212, 211,
31,
    235, 52, 44, 81, 234, 200, 72, 171, 242, 42, 104, 162, 253, 58, 206,
204,
    181, 112, 14, 86, 8, 12, 118, 18, 191, 114, 19, 71, 156, 183, 93,
135,
    21, 161, 150, 41, 16, 123, 154, 199, 243, 145, 120, 111, 157, 158,
178, 177,
    50, 117, 25, 61, 255, 53, 138, 126, 109, 84, 198, 128, 195, 189, 13,
87,
    223, 245, 36, 169, 62, 168, 67, 201, 215, 121, 214, 246, 124, 34,
185, 3,
    224, 15, 236, 222, 122, 148, 176, 188, 220, 232, 40, 80, 78, 51, 10,
74,
    167, 151, 96, 115, 30, 0, 98, 68, 26, 184, 56, 130, 100, 159, 38, 65,
    173, 69, 70, 146, 39, 94, 85, 47, 140, 163, 165, 125, 105, 213, 149,
59,
    7, 88, 179, 64, 134, 172, 29, 247, 48, 55, 107, 228, 136, 217, 231,
137,
    225, 27, 131, 73, 76, 63, 248, 254, 141, 83, 170, 144, 202, 216, 133,
97,
    32, 113, 103, 164, 45, 43, 9, 91, 203, 155, 37, 208, 190, 229, 108,
82,
    89, 166, 116, 210, 230, 244, 180, 192, 209, 102, 175, 194, 57, 75,
99, 182
};

constexpr std::array<std::uint8_t, 64> TAU = {
    0, 8, 16, 24, 32, 40, 48, 56,
    1, 9, 17, 25, 33, 41, 49, 57,
    2, 10, 18, 26, 34, 42, 50, 58,
    3, 11, 19, 27, 35, 43, 51, 59,
    4, 12, 20, 28, 36, 44, 52, 60,
    5, 13, 21, 29, 37, 45, 53, 61,
```

```
        6, 14, 22, 30, 38, 46, 54, 62,
        7, 15, 23, 31, 39, 47, 55, 63
    };

    constexpr std::array<std::uint64_t, 64> A = {
        0x8e20faa72ba0b470ULL, 0x47107ddd9b505a38ULL, 0xad08b0e0c3282d1cULL,
0xd8045870ef14980eULL,
        0x6c022c38f90a4c07ULL, 0x3601161cf205268dULL, 0x1b8e0b0e798c13c8ULL,
0x83478b07b2468764ULL,
        0xa011d380818e8f40ULL, 0x5086e740ce47c920ULL, 0x2843fd2067adea10ULL,
0x14aff010bdd87508ULL,
        0x0ad97808d06cb404ULL, 0x05e23c0468365a02ULL, 0x8c711e02341b2d01ULL,
0x46b60f011a83988eULL,
        0x90dab52a387ae76fULL, 0x486dd4151c3dfdb9ULL, 0x24b86a840e90f0d2ULL,
0x125c354207487869ULL,
        0x092e94218d243cbaULL, 0x8a174a9ec8121e5dULL, 0x4585254f64090fa0ULL,
0xaccc9ca9328a8950ULL,
        0x9d4df05d5f661451ULL, 0xc0a878a0a1330aa6ULL, 0x60543c50de970553ULL,
0x302a1e286fc58ca7ULL,
        0x18150f14b9ec46ddULL, 0x0c84890ad27623e0ULL, 0x0642ca05693b9f70ULL,
0x0321658cba93c138ULL,
        0x86275df09ce8aaa8ULL, 0x439da0784e745554ULL, 0xafc0503c273aa42aULL,
0xd960281e9d1d5215ULL,
        0xe230140fc0802984ULL, 0x71180a8960409a42ULL, 0xb60c05ca30204d21ULL,
0x5b068c651810a89eULL,
        0x456c34887a3805b9ULL, 0xac361a443d1c8cd2ULL, 0x561b0d22900e4669ULL,
0x2b838811480723baULL,
        0x9bcf4486248d9f5dULL, 0xc3e9224312c8c1a0ULL, 0xeffa11af0964ee50ULL,
0xf97d86d98a327728ULL,
        0xe4fa2054a80b329cULL, 0x727d102a548b194eULL, 0x39b008152acb8227ULL,
0x9258048415eb419dULL,
        0x492c024284fbaec0ULL, 0xaa16012142f35760ULL, 0x550b8e9e21f7a530ULL,
0xa48b474f9ef5dc18ULL,
        0x70a6a56e2440598eULL, 0x3853dc371220a247ULL, 0x1ca76e95091051adULL,
0x0edd37c48a08a6d8ULL,
        0x07e095624504536cULL, 0x8d70c431ac02a736ULL, 0xc83862965601dd1bULL,
0x641c314b2b8ee083ULL
    };

    constexpr std::array<std::string_view, 12> ROUND_CONSTANT_HEX = {
        "b1085bda1ecadae9ebcb2f81c0657c1f 2f6a76432e45d016714eb88d7585c4fc "
        "4b7ce09192676901a2422a08a460d315 05767436cc744d23dd806559f2a64507",
        "6fa3b58aa99d2f1a4fe39d460f70b5d7 f3feea720a232b9861d55e0f16b50131 "
        "9ab5176b12d699585cb561c2db0aa7ca 55dda21bd7cbcd56e679047021b19bb7",
        "f574dcac2bce2fc70a39fc286a3d8435 06f15e5f529c1f8bf2ea7514b1297b7b "
        "d3e20fe490359eb1c1c93a376062db09 c2b6f443867adb31991e96f50aba0ab2",
        "ef1fdfb3e81566d2f948e1a05d71e4dd 488e857e335c3c7d9d721cad685e353f "
        "a9d72c82ed03d675d8b71333935203be 3453eaa193e837f1220cbebc84e3d12e",
        "4bea6bacad4747999a3f410c6ca92363 7f151c1f1686104a359e35d7800fffbd "
        "bfcd1747253af5a3dfff00b723271a16 7a56a27ea9ea63f5601758fd7c6cfe57",
        "ae4faeae1d3ad3d96fa4c33b7a3039c0 2d66c4f95142a46c187f9ab49af08ec6 "
        "cffaa6b71c9ab7b40af21f66c2bec6b6 bf71c57236904f35fa68407a46647d6e",
        "f4c70e16eeaac5ec51ac86febf240954 399ec6c7e6bf87c9d3473e33197a93c9 "
        "0992abc52d822c3706476983284a0504 3517454ca23c4af38886564d3a14d493",
        "9b1f5b424d93c9a703e7aa020c6e4141 4eb7f8719c36de1e89b4443b4ddbc49a "
        "f4892bcb929b069069d18d2bd1a5c42f 36acc2355951a8d9a47f0dd4bf02e71e",
        "378f5a541631229b944c9ad8ec165fde 3a7d3a1b258942243cd955b7e00d0984 "
        "800a440bdbb2ceb17b2b8a9aa6079c54 0e38dc92cb1f2a607261445183235adb",
        "abbedea680056f52382ae548b2e4f3f3 8941e71cff8a78db1fffe18a1b336103 "
        "9fe76702af69334b7a1e6c303b7652f4 3698fad1153bb6c374b4c7fb98459ced",
        "7bcd9ed0efc889fb3002c6cd635afe94 d8fa6bbbebab07612001802114846679 "
        "8a1d71efea48b9caefbacd1d7d476e98 dea2594ac06fd85d6bcaa4cd81f32d1b",
        "378ee767f11631bad21380b00449b17a cda43c32bcdf1d77f82012d430219f9b "
        "5d80ef9d1891cc86e71da4aa88e12852 faf417d5d9b21b9948bc924af11bd720"
```

```
    };

    std::uint8_t hexDigit(char c)
    {
        if (c >= '0' && c <= '9') return static_cast<std::uint8_t>(c - '0');
        if (c >= 'a' && c <= 'f') return static_cast<std::uint8_t>(c - 'a' +
10);
        if (c >= 'A' && c <= 'F') return static_cast<std::uint8_t>(c - 'A' +
10);
        throw std::logic_error("Invalid hexadecimal constant");
    }

    Block fromHex(std::string_view text)
    {
        Block result{};
        std::size_t output = 0;
        int high = -1;

        for (char c : text)
        {
            if (c == ' ' || c == '\n' || c == '\r' || c == '\t') continue;
            const std::uint8_t value = hexDigit(c);
            if (high < 0)
                high = value;
            else
            {
                if (output >= result.size()) throw std::logic_error("Invalid
hexadecimal constant size");
                 result[output++] = static_cast<std::uint8_t>((high << 4) |
value);
                high = -1;
            }
        }

            if   (high   >=   0   ||   output   !=   result.size())   throw
std::logic_error("Invalid hexadecimal constant size");
        return result;
    }

    const std::array<Block, 12>& roundConstants()
    {
        static const std::array<Block, 12> constants = []
        {
            std::array<Block, 12> values{};
            for (std::size_t i = 0; i < values.size(); ++i)
                values[i] = fromHex(ROUND_CONSTANT_HEX[i]);
            return values;
        }();
        return constants;
    }

    void xorInto(Block& target, const Block& value)
    {
        for (std::size_t i = 0; i < target.size(); ++i) target[i] ^= value[i];
    }

    void lps(Block& state)
    {
        Block substituted{};
         for (std::size_t i = 0; i < state.size(); ++i) substituted[i] =
PI[state[i]];

        Block permuted{};
```

```
            for (std::size_t i = 0; i < state.size(); ++i) permuted[i] =
substituted[TAU[i]];

            Block transformed{};
            for (std::size_t group = 0; group < 8; ++group)
            {
                std::uint64_t word = 0;
                for (std::size_t byte = 0; byte < 8; ++byte)
                {
                    const std::uint8_t value = permuted[group * 8 + byte];
                    for (std::size_t bit = 0; bit < 8; ++bit)
                        if ((value & (0x80U >> bit)) != 0) word ^= A[byte * 8 +
bit];
                }
                for (std::size_t byte = 0; byte < 8; ++byte)
                   transformed[group * 8 + byte] = static_cast<std::uint8_t>(word
>> (56 - 8 * byte));
            }
            state = transformed;
        }

        Block gostEncrypt(const Block& key, const Block& message)
        {
            Block state = message;
            xorInto(state, key);

            const auto& constants = roundConstants();
            Block roundKey = key;
            for (std::size_t round = 0; round < constants.size(); ++round)
            {
                lps(state);
                xorInto(roundKey, constants[round]);
                lps(roundKey);
                xorInto(state, roundKey);
            }
            return state;
        }

        Block addBlocks(const Block& left, const Block& right)
        {
            Block result{};
            unsigned carry = 0;
            for (std::size_t i = result.size(); i-- > 0;)
            {
                const unsigned sum = static_cast<unsigned>(left[i]) + right[i] +
carry;
                result[i] = static_cast<std::uint8_t>(sum);
                carry = sum >> 8;
            }
            return result;
        }

        void addBits(Block& value, std::uint64_t addend)
        {
            for (std::size_t i = value.size(); i-- > 0 && addend != 0;)
            {
                    const unsigned sum = static_cast<unsigned>(value[i]) +
static_cast<unsigned>(addend & 0xffU);
                value[i] = static_cast<std::uint8_t>(sum);
                addend = (addend >> 8) + (sum >> 8);
            }
        }
```

```
      Block compress(const Block& hash, const Block& counter, const Block&
message)
      {
          Block key = hash;
          xorInto(key, counter);
          lps(key);

          Block result = gostEncrypt(key, message);
          xorInto(result, hash);
          xorInto(result, message);
          return result;
      }

      std::vector<std::uint8_t>    gostHash(const    std::vector<std::uint8_t>&
input, bool shortOutput)
      {
          Block hash{};
          if (shortOutput) hash.fill(1);
          Block counter{};
          Block sigma{};
          std::size_t offset = 0;

          while (input.size() - offset >= 64)
          {
              Block message{};
                                        std::reverse_copy(input.begin()       +
static_cast<std::ptrdiff_t>(offset),
                                                                input.begin()   +
static_cast<std::ptrdiff_t>(offset + 64),
                                  message.begin());
              hash = compress(hash, counter, message);
              addBits(counter, 512);
              sigma = addBlocks(sigma, message);
              offset += 64;
          }

          const std::size_t remainder = input.size() - offset;
          Block finalMessage{};
          finalMessage[63 - remainder] = 1;
                                   std::reverse_copy(input.begin()            +
static_cast<std::ptrdiff_t>(offset), input.end(),
                           finalMessage.begin() + static_cast<std::ptrdiff_t>(64
- remainder));

          hash = compress(hash, counter, finalMessage);
          addBits(counter, static_cast<std::uint64_t>(remainder) * 8);
          sigma = addBlocks(sigma, finalMessage);

          const Block zero{};
          hash = compress(hash, zero, counter);
          hash = compress(hash, zero, sigma);

          const std::size_t outputSize = shortOutput ? 32 : 64;
          std::vector<std::uint8_t> result(outputSize);

          for (std::size_t i = 0; i < outputSize; ++i)
              result[i] = hash[outputSize - 1 - i];
          return result;
      }

      std::vector<std::uint8_t> sha1(const std::vector<std::uint8_t>& input)
      {
          if (input.size() > std::numeric_limits<std::uint64_t>::max() / 8)
              throw std::length_error("Input is too large for SHA-1");
```

```
        std::vector<std::uint8_t> message = input;
                        const        std::uint64_t        bitLength        =
static_cast<std::uint64_t>(input.size()) * 8;
        message.push_back(0x80);
        while (message.size() % 64 != 56) message.push_back(0);
        for (int shift = 56; shift >= 0; shift -= 8)
           message.push_back(static_cast<std::uint8_t>(bitLength >> shift));

        std::array<std::uint32_t, 5> hash = {
            0x67452301U, 0xefcdab89U, 0x98badcfeU, 0x10325476U, 0xc3d2e1f0U
        };

        for (std::size_t offset = 0; offset < message.size(); offset += 64)
        {
            std::array<std::uint32_t, 80> words{};
            for (std::size_t i = 0; i < 16; ++i)
            {
                const std::size_t p = offset + i * 4;
                words[i] = (static_cast<std::uint32_t>(message[p]) << 24)
                         | (static_cast<std::uint32_t>(message[p + 1]) << 16)
                         | (static_cast<std::uint32_t>(message[p + 2]) << 8)
                         | static_cast<std::uint32_t>(message[p + 3]);
            }
            for (std::size_t i = 16; i < words.size(); ++i)
                words[i] = rotateLeft(words[i - 3] ^ words[i - 8] ^ words[i
- 14] ^ words[i - 16], 1);

            std::uint32_t a = hash[0];
            std::uint32_t b = hash[1];
            std::uint32_t c = hash[2];
            std::uint32_t d = hash[3];
            std::uint32_t e = hash[4];

            for (std::size_t i = 0; i < 80; ++i)
            {
                std::uint32_t function = 0;
                std::uint32_t constant = 0;
                if (i < 20)
                {
                    function = (b & c) | (~b & d);
                    constant = 0x5a827999U;
                }
                else if (i < 40)
                {
                    function = b ^ c ^ d;
                    constant = 0x6ed9eba1U;
                }
                else if (i < 60)
                {
                    function = (b & c) | (b & d) | (c & d);
                    constant = 0x8f1bbcdcU;
                }
                else
                {
                    function = b ^ c ^ d;
                    constant = 0xca62c1d6U;
                }

                const std::uint32_t temp = rotateLeft(a, 5) + function + e +
constant + words[i];
                e = d;
                d = c;
                c = rotateLeft(b, 30);
```

```
                b = a;
                a = temp;
            }

            hash[0] += a;
            hash[1] += b;
            hash[2] += c;
            hash[3] += d;
            hash[4] += e;
        }

        std::vector<std::uint8_t> result;
        result.reserve(20);
        for (std::uint32_t word : hash)
            for (int shift = 24; shift >= 0; shift -= 8)
                result.push_back(static_cast<std::uint8_t>(word >> shift));
        return result;
    }

    std::string toHex(const std::vector<std::uint8_t>& bytes)
    {
        std::string result;
        result.reserve(bytes.size() * 2);
        for (std::uint8_t byte : bytes)
        {
            constexpr char digits[] = "0123456789abcdef";
            result.push_back(digits[byte >> 4]);
            result.push_back(digits[byte & 0x0f]);
        }
        return result;
    }

    int main(int argc, char* argv[])
    {
        if (argc != 3)
        {
             std::cerr << "Usage: " << argv[0] << " <sha1|gost256|gost512>
<input_file>\n";
            return 1;
        }

        const std::string algorithm = argv[1];
         if (algorithm != "sha1" && algorithm != "gost256" && algorithm !=
"gost512")
        {
              std::cerr << "Error: algorithm must be sha1, gost256, or
gost512\n";
            return 1;
        }

        std::ifstream file(argv[2], std::ios::binary);
        if (!file)
        {
            std::cerr << "Error: failed to open input file\n";
            return 1;
        }

        std::vector<std::uint8_t> input;
        for (char value; file.get(value);)
            input.push_back(static_cast<std::uint8_t>(static_cast<unsigned
char>(value)));
        if (file.bad())
        {
            std::cerr << "Error: failed to read input file\n";
```

```
            return 1;
        }

        const std::vector<std::uint8_t> digest = algorithm == "sha1"
            ? sha1(input)
            : gostHash(input, algorithm == "gost256");

        if (algorithm == "sha1")
            std::cout << "SHA-1: " << toHex(digest) << '\n';
        else if (algorithm == "gost256")
             std::cout << "GOST 34.11-2012 (256-bit): " << toHex(digest) <<
'\n';
        else
             std::cout << "GOST 34.11-2012 (512-bit): " << toHex(digest) <<
'\n';
        return 0;
    }
```